

On the Distribution of DTN Reachability Information: A Quantitative Push-vs-Pull Analysis

Juan A. Fraire, *Senior Member, IEEE*, Tobias Nöthlich, Marius Feldmann, and Felix Walter

Abstract—As space networks scale from isolated point-to-point links to multi-operator, multi-hop infrastructures—spanning cislunar relays, Mars surface meshes, and deep-space probes—nodes must discover how to reach each other’s Bundle Protocol endpoints. Today, this Endpoint Identifier (EID) to Convergence-Layer Adapter (CLA) binding is pre-provisioned; it will not scale. Two Internet paradigms offer competing solutions: proactive flooding (*à la* BGP) that replicates every binding everywhere, and on-demand resolution (*à la* DNS) that queries an authority only when needed. We build OMNeT++ models of both paradigms and evaluate them across 13 experiments on synthetic grids (up to 400 nodes, 500 EIDs) and three realistic scenarios—terrestrial disaster response, lunar Artemis, and Mars exploration—covering one-way delays from 5 ms to 12 min. The results reveal a sharp, delay-driven regime map. Below ~ 100 ms round-trip time, DNS delivers $9\text{--}141\times$ lower overhead with sub-200 ms query latency; above ~ 3 s, BGP’s one-time convergence cost amortizes after a single query, making it the only viable option at Mars distances where each DNS lookup takes minutes. A contested crossover emerges at cislunar delays ($\sim 1\text{--}3$ s), where neither protocol dominates, and a hybrid strategy may be optimal. Under EID churn, BGP maintains 100% accuracy at all rates, while DNS degrades to 30% when the churn interval falls below the cache TTL. These findings provide the first systematic, quantitative guidelines for choosing—or combining—push and pull EID resolution in networks that span the solar system.

Index Terms—Delay-tolerant networking, endpoint identifier resolution, BGP, DNS, space networking, cislunar communications, interplanetary Internet

I. INTRODUCTION

Every packet on the Internet relies on two infrastructures so mature they are almost invisible: BGP floods reachability to every backbone router so that forwarding is instantaneous, while DNS resolves names on demand so that no router need store every hostname. Together, they embody a decades-old design tension—*push everything now* versus *pull only when needed*—that the terrestrial Internet resolved pragmatically: routing tables are pushed, name bindings are pulled.

Space networking is reopening this question. The Delay-Tolerant Networking (DTN) architecture [1], [2] introduces Bundle Protocol Endpoint Identifiers (EIDs) that must be mapped to Convergence-Layer Adapter (CLA) addresses before any bundle can be forwarded—the exact analogue of IP routing and DNS resolution. Yet the DTN standards deliberately leave this binding unspecified [3], and current deployments rely on static, pre-provisioned tables. As NASA’s Artemis program [4] and future Mars architectures [5] scale to

dozens of heterogeneous, mobile nodes operated by multiple agencies, static configuration becomes untenable.

Recent proposals have begun to fill this gap from both directions. Feldmann et al. [6] extend BGP UPDATE messages to carry DTN EID reachability, leveraging BGP’s path-vector machinery for loop-free flooding. The original scope of this work targets *edge BPA configuration*: how a Bundle Protocol Agent is configured by its immediate gateway—analogue to how a DSL modem receives routing information from its telco provider—rather than flooding EID bindings across a general multi-hop mesh. Kline [7] proposes the `ipn.arpa` DNS zone to resolve `ipn`-scheme EIDs through the existing DNS hierarchy. Both approaches are technically sound, but they inherit fundamentally different cost structures: BGP pays $O(N)$ messages per EID change for $O(1)$ lookups; DNS pays $O(1)$ per registration but $O(D \cdot d)$ latency per query, where D is the network diameter and d the link delay. In terrestrial networks, where d is milliseconds, this difference is negligible. In space, where d ranges from 1.3 s (Earth–Moon) to 12 min (Earth–Mars), the choice is existential.

No prior work has systematically compared push and pull EID resolution across the full DTN delay spectrum as a *general* multi-hop problem. This paper closes that gap—deliberately broadening the scope beyond the edge-gateway configuration originally targeted by [6]—with three contributions:

- 1) **Simulation models.** We develop OMNeT++ implementations of a BGP-like path-vector protocol and a DNS-like query-response protocol with caching, operating over identical topologies for a fair comparison. Both models, along with all experiment configurations, are publicly available [8].
- 2) **Comprehensive evaluation.** We present 13 experiments spanning synthetic grids (25–400 nodes, 10–500 EIDs, 10 ms–20 s link delays) and three realistic hierarchical deployments—terrestrial disaster response (22 nodes), lunar Artemis (12 nodes), and Mars exploration (10 nodes)—covering churn, caching, and scalability dimensions.
- 3) **Delay-driven regime map.** We identify three deployment regimes with quantitative boundaries: terrestrial (< 100 ms RTT), where DNS saves $9\text{--}141\times$ in overhead; deep-space (> 3 s RTT), where BGP is the only viable option; and a contested cislunar band in between, where hybrid strategies are needed.

The remainder of this paper is organized as follows. Section II reviews DTN, BGP, and DNS background and identifies the EID resolution gap. Section III develops simulation mod-

J. A. Fraire is with Inria, INSA Lyon, CITI, UR3720, 69621 Villeurbanne, France, and also with CONICET – Universidad Nacional de Córdoba, Córdoba, Argentina and D3TN GmbH.

T. Nöthlich, M. Feldmann and F. Walter are with D3TN GmbH.

els, derives complexity bounds, and formulates four testable hypotheses. Section IV evaluates these hypotheses through 13 experiments. Section V synthesizes deployment recommendations for each point in the delay spectrum.

II. BACKGROUND

This section reviews the three pillars underpinning our work: Delay-Tolerant Networking for space communications, the Internet’s inter-domain routing infrastructure (BGP), and its naming system (DNS). We then identify the architectural gap that motivates their adaptation to DTN environments.

A. Delay-Tolerant Networking in Space

The concept of an “Interplanetary Internet” was first articulated by Cerf et al. [9], recognizing that the TCP/IP assumptions of low latency, continuous connectivity, and symmetric paths do not hold in space environments. Fall [1] formalized the Delay-Tolerant Networking (DTN) architecture, introducing a store-carry-forward overlay that tolerates intermittent links, high delays, and asymmetric data rates.

The DTN architecture centers on the Bundle Protocol (BP), standardized as RFC 5050 [10] and subsequently updated to BPv7 in RFC 9171 [2]. BP operates as an overlay above heterogeneous lower-layer protocols, using *Convergence-Layer Adapters* (CLAs) to interface with the underlying transport—whether TCP [11], UDP, LTP, or radio links. Each bundle-layer endpoint is identified by an *Endpoint Identifier* (EID), a URI-style name (e.g., `ipn:13.1` or `dtn://mars-hab1/telemetry`) that must be resolved to a concrete CLA address before a bundle can be forwarded.

Space agencies have increasingly adopted DTN for operational missions. The Consultative Committee for Space Data Systems (CCSDS) has standardized BP for space use [12], and DTN has been demonstrated on the International Space Station, lunar relay scenarios, and deep-space testbeds [13]. NASA’s Artemis program envisions a cislunar communications infrastructure (LunaNet) [4] where heterogeneous surface assets, orbital relays, and Earth ground stations must interoperate—a setting that demands scalable EID discovery. Looking further, Mars exploration architectures [5] will face propagation delays of 3–22 minutes (one-way), making any synchronous protocol inherently challenging.

A critical open problem in these deployments is *EID-to-CLA resolution*: how does a node discover which CLA endpoint to contact for a given EID? In small, static networks, this mapping can be pre-configured. However, as DTN deployments scale to tens or hundreds of mobile nodes with dynamic services, a distributed, automated resolution mechanism becomes essential [14]. This is the space-networking analogue of the IP layer’s routing and naming functions—precisely the roles that BGP and DNS serve in the terrestrial Internet.

A key architectural distinction is that DTN employs *late binding*: the resolution of a bundle’s destination EID into a concrete next-hop CLA address is deferred until forwarding time, rather than being fixed when the bundle is created [1]. In IP networks, destination-to-next-hop resolution occurs immediately against a relatively stable routing table. In DTN, a

bundle carries only a destination EID; the full mapping—EID $\rightarrow$ reachable node $\rightarrow$ next scheduled contact $\rightarrow$ convergence-layer peer—is resolved at each hop when the node actually forwards the bundle, possibly long after its creation. Late binding is essential because contacts in space networks are intermittent and time-varying: the “right” next hop depends on which links are available at the moment of forwarding. If DTN is viewed as a delay-tolerant overlay interconnecting islands of conventional (non-delay-tolerant) networks, then at each island—where IP connectivity is available—a service such as BGP or DNS can assist in resolving the next-hop CLA parameters within that local domain. The push-vs-pull question studied in this paper therefore concerns how these per-domain resolution services should operate under the extreme and heterogeneous delays characteristic of space DTN deployments.

B. BGP: Inter-Domain Routing in the Internet

The Border Gateway Protocol (BGP-4) [15] is the de facto standard for inter-domain routing on the Internet, enabling over 75,000 autonomous systems (ASes) to exchange reachability information for IP prefixes. BGP is a *path-vector* protocol: each route advertisement carries the full AS path, enabling loop detection and policy-based route selection.

BGP operates on a *push* model. When a network originates or learns a new prefix, it proactively announces the route to all of its BGP peers, which in turn propagate the announcement (with their own AS appended to the path) to their peers, and so on. This flooding process continues until all BGP speakers in the network have a consistent view of reachability—a state known as *convergence*. Convergence times in the Internet range from seconds for simple announcements to minutes for complex withdrawal events [16].

The key strengths of BGP’s push model are *instant lookup* (once converged, any prefix can be resolved from the local routing table without network communication) and *strong consistency* (all routers converge to the same view). The weaknesses are *state explosion* (every router stores the full routing table, currently ~ 1 million entries) and *update storms* (each prefix change triggers network-wide propagation). In the DTN context, these characteristics are amplified: convergence time scales with propagation delay ($D \cdot d$ for diameter D), but post-convergence lookups remain instantaneous.

Recent work has proposed adapting BGP for DTN EID distribution [6], extending BGP UPDATE messages with DTN-specific attributes to carry EID reachability information. In this approach, BGP’s message format and session model are reused to deliver EID bindings from a gateway to its attached BPA—not for transitive, network-wide propagation, but for the single-hop edge interface at which a BPA is configured by its upstream provider. The approach inherits BGP’s overhead characteristics, an important consideration when links are bandwidth-constrained or intermittent.

A key distinction must be drawn regarding the *intended application domain* of this BGP extension. Its core use case is the *configuration of end-user-facing BPAs by their gateways*: a BPA at the edge of a DTN domain learns EID reachability

from its directly connected gateway, much as a DSL router receives prefix information from its telco provider. In this edge-gateway model, the path-vector properties serve to configure local forwarding state at a single administrative boundary, not to flood bindings transitively across a multi-hop network. The present paper generalizes this push paradigm to multi-hop DTN deployments in order to enable a systematic, head-to-head comparison with the DNS pull approach—a deliberate broadening of scope. Readers should therefore interpret our overhead and latency results for the BGP model as characterizing this *generalized* multi-hop flooding behavior; conclusions about BGP’s applicability in terrestrial regimes do not directly invalidate the original, narrower edge-gateway use case.

C. DNS: The Internet’s Naming System

The Domain Name System (DNS) [17] provides the Internet’s hierarchical, distributed naming infrastructure, resolving human-readable domain names to IP addresses. Unlike BGP’s push model, DNS operates on a *pull* (on-demand) model: name resolution occurs only when a client issues a query.

DNS resolution follows a hierarchical delegation chain. A client’s *recursive resolver* queries authoritative name servers on the client’s behalf, traversing the hierarchy from root servers to top-level domain servers to the authoritative server for the queried name. To reduce latency and authority-server load, resolvers *cache* responses for a duration specified by the record’s *Time-to-Live* (TTL). Caching is central to DNS’s scalability: studies show that caching reduces authority queries by 60–90% under typical web traffic patterns [18], [19].

DNS’s pull model offers two advantages over push approaches: *minimal background overhead* (no traffic is generated until a name is actually queried) and *centralized state* (only authoritative servers and caches store records, not every node in the network). The primary weakness is *per-query latency*: every cache miss incurs a round-trip to the authoritative server. In terrestrial networks, this round-trip is typically under 100 ms. In space networks, it can take seconds (cislunar) or minutes (interplanetary), fundamentally changing the cost calculus.

The adaptation of DNS to DTN has been explored through the `ipn.arpa` zone proposal [7], which maps `ipn`-scheme EIDs to DNS resource records. This enables standard DNS infrastructure to serve as the resolution back-end for DTN EIDs, potentially allowing space networks to reuse the operational tooling and expertise of the global DNS ecosystem.

One might ask whether DNS *zone replication*—distributing synchronized copies of the authority database to remote sites via zone transfer—could eliminate per-query latency by serving lookups from a local replica. However, this would transform DNS into a push model: every EID binding change must be propagated across interplanetary links to keep all replicas current, reintroducing the network-wide synchronization overhead that characterizes BGP. The push-versus-pull tradeoff is therefore a paradigm-level distinction, not a technological one: any mechanism that guarantees low-latency local lookups at remote sites must proactively distribute state—whether via BGP UPDATE messages or DNS zone transfers. Furthermore,

deeply remote networks such as a Mars surface mesh are likely to constitute a separate DTN administrative domain with locally managed EID namespaces, where most queries are served by local authorities without any interplanetary round-trip.

D. The EID Resolution Gap

The DTN architecture defines the bundle-layer addressing (EIDs) and the convergence-layer transport (CLAs) but leaves the *binding* between them deliberately unspecified [3]. In practice, EID-to-CLA mappings are typically pre-provisioned or managed through out-of-band configuration—an approach that does not scale to dynamic, multi-operator space networks.

Both BGP and DNS are natural candidates for filling this gap: BGP’s path-vector flooding can distribute EID reachability much as it distributes IP prefix reachability, while DNS’s hierarchical query-response model can resolve EIDs much as it resolves domain names. However, the two paradigms embody fundamentally different trade-offs between overhead, latency, and consistency—and these trade-offs are amplified by the extreme and heterogeneous propagation delays characteristic of space networks.

No prior work has systematically compared push and pull approaches for EID resolution across the full DTN delay spectrum, from terrestrial (milliseconds) through cislunar (seconds) to interplanetary (minutes). The remainder of this paper addresses this gap.

III. MODEL

This section presents the simulation models for comparing proactive (BGP-like) and reactive (DNS-like) approaches for distributing Endpoint Identifier (EID) reachability information in Delay-Tolerant Networks (DTNs). The two paradigms embody a fundamental design tension: *push* protocols pay an upfront cost to replicate state everywhere, enabling instant local lookups; *pull* protocols avoid replication overhead but pay a per-query round-trip cost. In terrestrial networks, this tradeoff is well understood, but DTN environments—where link delays range from milliseconds to minutes—shift the balance in ways that have not been systematically quantified. Both protocols are implemented in OMNeT++ [20] and operate over identical physical topologies to ensure a fair comparison; the simulation code is publicly available in our repository [8]. Fig. 1 illustrates the two paradigms side by side.

A. System Model

We consider a DTN consisting of N nodes interconnected by links with propagation delay d_{ij} between nodes i and j . Each node may host Bundle Protocol (BP) agents identified by unique EIDs [2]. The fundamental problem is: *how should nodes discover the Convergence-Layer Adapter (CLA) endpoint responsible for a given EID?*

Let $\mathcal{E} = \{e_1, e_2, \dots, e_M\}$ denote the set of EIDs in the network. For each EID e_k , there exists exactly one authoritative endpoint binding:

$$\text{endpoint}(e_k) = \langle \text{nodeId}, \text{claProtocol}, \text{port} \rangle \quad (1)$$

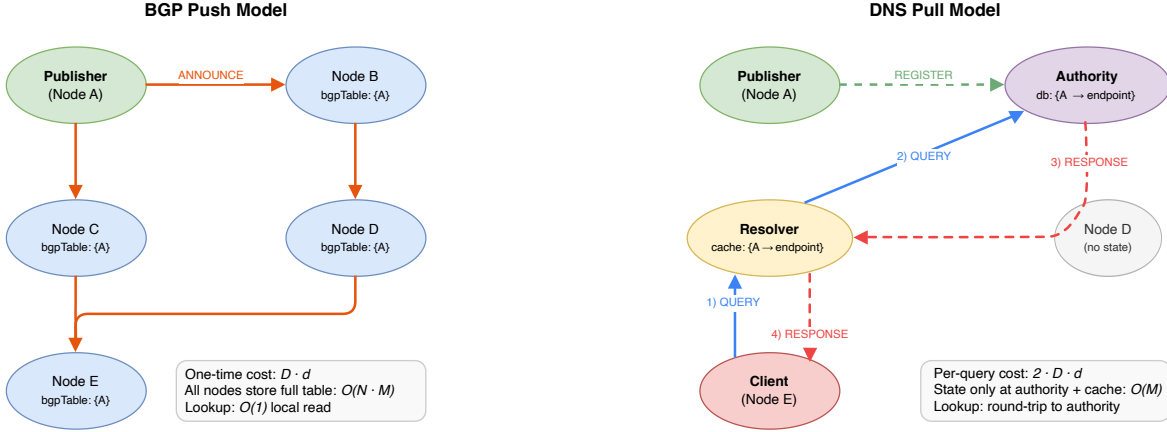


Fig. 1. Push vs. pull EID resolution. *Left:* BGP-like flooding replicates every binding to every node (one-time cost $D \cdot d$, $O(N \cdot M)$ network state, $O(1)$ lookup). *Right:* DNS-like resolution stores bindings at an authority and resolves on demand (per-query cost $2 \cdot D \cdot d$, $O(M)$ centralized state). Grey nodes hold no EID state under DNS.

The goal is to enable any node i to resolve $\text{endpoint}(e_k)$ optimizing three potentially conflicting objectives:

- **Correctness:** The resolved endpoint matches the current authoritative binding, even under churn (EID migrations and withdrawals)
- **Timeliness:** Resolution completes within acceptable latency bounds, both for first discovery and subsequent lookups
- **Efficiency:** Minimal network overhead and distributed state, particularly important in bandwidth-constrained DTN links

As we show in the complexity analysis below, no single protocol dominates all three objectives simultaneously. The push model favors correctness and timeliness at the expense of efficiency, while the pull model favors efficiency at the expense of per-query timeliness. The relative importance of each objective depends critically on the propagation delay regime.

Nodes in the network can assume different roles:

- **Publishers:** Nodes that originate EID bindings and announce them to the network
- **Clients:** Nodes that need to resolve EIDs to their endpoint bindings
- **Resolvers** (DNS only): Caching intermediaries that handle client queries
- **Authorities** (DNS only): Nodes that store authoritative EID records

B. Network Model

We employ two classes of topologies. *Synthetic grids* provide controlled, parameterized environments for isolating individual factors (network size, EID count, churn rate, link delay). *Hierarchical topologies* model realistic deployments where the protocol choice has direct operational consequences.

1) *Synthetic Grid Topology:* The primary experimental topology is a two-dimensional grid network:

$$G = (V, E) \text{ where } |V| = w \times h \quad (2)$$

Nodes are indexed $v_{i,j}$ for $i \in [0, w - 1]$, $j \in [0, h - 1]$, with edges connecting adjacent nodes:

$$E = \{(v_{i,j}, v_{i',j'}) : |i - i'| + |j - j'| = 1\} \quad (3)$$

The network diameter (maximum shortest path) for a $w \times h$ grid is $D = w + h - 2$. For a 10×10 grid, $D = 18$ hops. We vary grid size from 5×5 to 20×20 to study scalability, and sweep link delay from $d = 10$ ms (terrestrial) through $d \in \{1, 5, 10, 20\}$ s (deep-space) to characterize behavior across the latency spectrum.

All links are modeled as delay channels with configurable propagation delay d . The channel model abstracts away transmission time, focusing on propagation delay, which is appropriate for control-plane traffic with small message sizes.

2) *Realistic Hierarchical Topologies:* To validate findings from the grid experiments, we model three hierarchical topologies spanning the DTN latency spectrum: terrestrial disaster response (22 nodes, 5–30 ms delays), lunar exploration (12 nodes, 5 ms to 1.3 s delays), and Mars exploration (10 nodes, 1 s to 720 s delays). These topologies include specialized channel types such as Earth-Moon links (1.3 s one-way light delay), Mars orbital relay channels (60 s), and interplanetary channels (720 s average). The scenarios are detailed in Section IV-F.

An important modeling assumption applies to all three realistic scenarios: all links are modeled as *stable, always-on delay channels*—there are no link disruptions, contact schedules, or intermittency. This is a deliberate simplification that isolates the *control-plane overhead question* (how much traffic does each protocol generate to distribute EID bindings?) from the *disruption-tolerance question* (how does DTN handle link outages?). The results therefore characterize protocol behavior given *established connectivity*, not during link failures. Real deployments—especially disaster response and deep-space networks—will involve intermittent contacts and scheduled link windows; integrating contact-plan-aware routing with EID resolution is left as future work (see Section V).

C. BGP-like Push Model

The BGP-like model implements a path-vector routing protocol inspired by BGP-4 [15]. EID bindings are proactively flooded to all nodes upon publication, ensuring that each node maintains a local routing table with all known EIDs.

1) *Data Structures*: Each node maintains a routing table indexed by EID:

$$\text{bgpTable} : \mathbb{Z}^+ \rightarrow \text{BgpEntry} \quad (4)$$

where each entry contains:

- **eid**: The Endpoint Identifier
- **originId**: Publishing node ID
- **nextHop**: Gate index for forwarding
- **pathLength**: Distance in hops from the origin
- **version**: Update sequence number for churn handling
- **endpoint**: CLA binding information
- **path**: Full path vector for loop detection

2) *Message Types*: The protocol defines two message types:

- **BgpAnnounce** (64 + 4|path| bytes): Propagates EID reachability with path vector
- **BgpWithdraw** (32 bytes): Removes EID reachability

3) *Publication Algorithm*: When node p publishes EID e with endpoint ϵ :

- 1) Create local table entry with $\text{pathLength} = 0$ and $\text{path} = [p]$
- 2) Increment version: $v \leftarrow \text{publishedVersions}[e] + 1$
- 3) Construct announcement: $m \leftarrow \text{BgpAnnounce}(e, p, 0, v, \epsilon, t_{\text{now}}, [p])$
- 4) Flood to all neighbors: $\forall g \in \text{neighborGates} : \text{send}(m, g)$

4) *Announcement Processing*: Upon receiving announcement m at node n from gate g_{in} :

Step 1: Loop Detection. If $n \in m.\text{path}$, discard the message.

Step 2: Path Extension. Compute $\text{path}' \leftarrow m.\text{path} \cup \{n\}$ and $\text{pathLength}' \leftarrow m.\text{pathLength} + 1$.

Step 3: Acceptance Decision. Let $\text{existing} = \text{bgpTable}[m.\text{eid}]$. Accept if any condition holds:

$$\text{accept}(m) = \begin{cases} \text{true} & \text{if } \nexists \text{existing} \\ \text{true} & \text{if } m.\text{originId} = \text{existing}.\text{originId} \\ & \quad \wedge m.\text{version} > \text{existing}.\text{version} \\ \text{true} & \text{if } \text{pathLength}' < \text{existing}.\text{pathLength} \\ \text{true} & \text{if } \text{pathLength}' = \text{existing}.\text{pathLength} \\ & \quad \wedge m.\text{originId} < \text{existing}.\text{originId} \\ \text{false} & \text{otherwise} \end{cases} \quad (5)$$

Step 4: Table Update and Forwarding. If accepted, update the local table and forward to all neighbors except the arrival gate.

5) *Withdrawal Processing*: Withdrawal w for EID e is processed only if $w.\text{originId}$ matches the current entry's origin and $w.\text{version} \geq \text{existing}.\text{version}$. Upon acceptance, the entry is removed, and the withdrawal is forwarded.

6) *Complexity Analysis*: **Convergence Time**: For a network with diameter D and link delay d :

$$T_{\text{conv}}^{\text{BGP}} = D \cdot d \quad (6)$$

For a 10×10 grid with $d = 10$ ms: $T_{\text{conv}}^{\text{BGP}} = 18 \times 0.01 = 0.18$ s. Crucially, this cost is paid *once* per EID publication; all subsequent lookups are instantaneous local table reads.

State Complexity: Per-node state is $O(M)$ where M is the number of EIDs. Network-wide state is $O(N \cdot M)$ —full replication. This implies that overhead should scale super-linearly with network size (each announcement traverses more hops and reaches more nodes) and linearly with EID count.

Message Complexity: $O(N)$ messages per EID publication in a grid topology. Under churn with interval Δ_{churn} , the sustained overhead rate is $O(N \cdot M / \Delta_{\text{churn}})$, which we expect to become the dominant cost in high-churn scenarios.

D. DNS-like Pull Model

The DNS-like model implements an on-demand resolution protocol inspired by the Domain Name System [17]. EID bindings are stored at authoritative servers and resolved via queries with optional caching.

1) *Data Structures*: **Authority Record** (at authority nodes):

- **eid**: The Endpoint Identifier
- **publisherId**: Publisher node ID
- **endpoint**: CLA binding information
- **ttl**: Time-to-live in seconds
- **version**: Update sequence number
- **lastChangedTime**: Timestamp of last modification

Cache Entry (at resolver nodes):

- **eid**: The Endpoint Identifier
- **endpoint**: Cached CLA binding
- **ttl**: Original TTL value
- **expiryTime**: When the cache entry expires
- **recordTime**: Authority's lastChangedTime (for staleness detection)

2) *Message Types*:

- **DnsQuery** (48 bytes): Client query for an EID
- **DnsResponse** (80 bytes): Response with endpoint binding
- **DnsRegister** (72 bytes): Publisher registration at authority
- **DnsDeregister** (24 bytes): Publisher withdrawal

3) *Registration*: When publisher p registers EID e at authority a :

If $p = a$ (local authority), the record is stored directly. Otherwise, a **DnsRegister** message is sent to the authority.

Authority assignment uses modular hashing:

$$a = (e \bmod |\mathcal{A}|) + a_{\text{first}} \quad (7)$$

where $\mathcal{A}$ is the set of authority nodes.

4) *Query Resolution*: Client-initiated query for EID e :

- 1) **Local Cache Check**: If $e \in \text{dnsCache}$ and not expired, return cache hit
- 2) **Cache Miss**: Create query and forward to resolver
- 3) **Resolver Processing**: Check resolver cache; if miss, forward to authority
- 4) **Authority Processing**: Look up record and return response
- 5) **Response Path**: Response traverses back, with resolvers caching the result

5) *Caching Mechanism*: Cache entries are inserted with expiry time $t_{\text{now}} + \tau$ where τ is the TTL. When the cache exceeds capacity, expired entries are removed first, then the entry with the earliest expiry time is evicted.

The cache hit rate is:

$$\text{hitRate} = \frac{\sum \text{cacheHits}}{\sum \text{cacheHits} + \sum \text{cacheMisses}} \quad (8)$$

6) *Complexity Analysis*: **Query Latency (cache miss)**:

$$L_{\text{query}} \approx 2 \cdot D \cdot d \quad (9)$$

This per-query cost is exactly $2\times$ the BGP convergence time—but unlike BGP, it is paid on *every* uncached resolution. For Q queries to distinct EIDs, the cumulative DNS latency is $Q \cdot 2Dd$, whereas BGP pays a fixed Dd regardless of Q . We therefore expect a break-even point at $Q^* = 1/2$, i.e., BGP amortizes its convergence cost after a single query in the worst case.

State Complexity: Authority state is $O(M)$. Resolver cache is $O(\min(M, C))$ where C is cache capacity. Network-wide state is $O(M)$ —centralized. Unlike BGP, DNS overhead should be independent of network size N .

Message Complexity: $O(1)$ messages per registration; $O(1)$ per query. Under churn, the authority database is updated locally with no network-wide propagation, so DNS churn overhead should be negligible compared to BGP. However, cached entries become stale, introducing a freshness–efficiency tradeoff governed by the TTL parameter τ .

E. Fair Comparison Design

A naive comparison would be unfair: DNS in practice operates over an already routed IP network, whereas BGP-like flooding operates at the network layer. If DNS queries had to be routed hop-by-hop over a not-yet-converged network, DNS would be penalized for a problem orthogonal to our research question. To isolate the push-vs-pull comparison from routing convergence effects, we implement a “direct mode” for DNS.

In direct mode (`dnsDirectMode=true`), DNS queries use OMNeT++’s `sendDirect()` mechanism with RTT-based delays:

$$\text{RTT}_{\text{DNS}} = 2 \cdot D \cdot d \quad (10)$$

This models DNS running over a pre-routed IP underlay, ensuring that:

- BGP announcements propagate through the physical topology hop-by-hop

- DNS queries logically traverse: Client $\rightarrow$ Resolver $\rightarrow$ Authority $\rightarrow$ Resolver $\rightarrow$ Client, with delay proportional to network diameter
- The comparison isolates the *information distribution strategy* (push vs. pull) from lower-layer routing concerns

F. Staleness and Accuracy Model

A central `GroundTruth` module maintains authoritative records for all EIDs, enabling correctness verification without protocol bias.

1) *BGP Accuracy*: A node’s BGP entry for EID e is *correct* if:

$$\text{correct}_{\text{BGP}}(n, e) = \begin{cases} \text{bgpTable}[e].\text{endpoint} & \text{if active} \\ = \text{GT}[e].\text{endpoint} & \\ e \notin \text{bgpTable} & \text{if withdrawn} \end{cases} \quad (11)$$

Network-wide BGP accuracy:

$$\text{Accuracy}_{\text{BGP}}(e) = \frac{|\{n : \text{correct}_{\text{BGP}}(n, e)\}|}{N} \quad (12)$$

Because BGP propagates every UPDATE incrementally, we expect $\text{Accuracy}_{\text{BGP}} = 1$ once the network has converged, regardless of subsequent churn—each update is immediately flooded.

2) *DNS Staleness*: A DNS response is *stale* if the cached record predates the last authoritative change:

$$\text{stale}(r) = r.\text{recordTime} < \text{GT}[r.\text{eid}].\text{lastChangeTime} \quad (13)$$

Under churn with interval Δ_{churn} :

$$P(\text{stale}) \approx \frac{\Delta_{\text{churn}}}{\tau} \text{ for } \tau > \Delta_{\text{churn}} \quad (14)$$

where τ is the TTL. This predicts that DNS accuracy degrades as churn rate increases relative to TTL, creating a fundamental trade-off: shorter TTLs improve freshness but increase query load on the authoritative name server.

3) *Convergence Detection*: Convergence for EID e occurs when $\geq 99\%$ of nodes have correct information:

$$\text{converged}(e, t) = \frac{|\{n : \text{correct}(n, e, t)\}|}{N} \geq 0.99 \quad (15)$$

We distinguish *initial convergence* (first publication) from *churn convergence* (subsequent updates). A critical regime arises when the *convergence-to-churn ratio* $R = T_{\text{conv}}/\Delta_{\text{churn}}$ exceeds 1, meaning the network cannot fully converge between successive changes. We hypothesize that BGP maintains high accuracy even when $R > 1$ (because updates propagate incrementally), while DNS accuracy degrades proportionally to R .

G. Query Distribution Model

Query patterns follow Zipf’s law [19]:

$$P(\text{query for EID } e_k) \propto \frac{1}{k^\alpha} \quad (16)$$

where k is the rank ($1 = \text{most popular}$) and α is the skewness parameter. Values of $\alpha = 0$ yield uniform distribution, while $\alpha = 1.0$ approximates typical web traffic patterns.

TABLE I
SIMULATION METRICS

Category	Metric	Unit
Overhead	Total bytes sent	bytes
	Total messages	count
Latency	Discovery latency	seconds
	Convergence time	seconds
	Query latency	seconds
Accuracy	BGP accuracy	[0, 1]
	DNS accuracy	[0, 1]
	Stale rate	[0, 1]
State	BGP table size	entries
	DNS cache size	entries

TABLE II
PROTOCOL COMPLEXITY COMPARISON

Aspect	BGP (Push)	DNS (Pull)
State per node	$O(M)$	$O(1)$ client, $O(C')$ resolver
Network state	$O(N \cdot M)$	$O(M)$
Publication cost	$O(N)$ messages	$O(1)$ messages
Query cost	$O(1)$ (local)	$O(D)$ hops
Churn cost	$O(N)$ per change	$O(1)$ per change
Convergence time	$O(D \cdot d)$	N/A (on-demand)
Discovery latency	$D \cdot d$	$2 \cdot D \cdot d$
Resolution latency	≈ 0 (local)	$2 \cdot D \cdot d$

H. Metrics Summary

Table I summarizes the key metrics collected during simulation.

I. Complexity Comparison and Hypotheses

Table II compares the theoretical complexity of both approaches. These asymptotic differences lead to four testable hypotheses about protocol performance across the DTN delay spectrum.

The complexity comparison reveals that the two protocols optimize for different regimes. BGP's $O(N \cdot M)$ state replication and $O(N)$ per-change churn cost become increasingly expensive with network size and dynamism, favoring DNS. Conversely, DNS's $O(D \cdot d)$ per-query cost grows with propagation delay, increasingly favoring BGP. This analysis motivates the following hypotheses, which we evaluate experimentally in Section IV:

- H1:** *DNS dominates in low-latency regimes.* When the per-query RTT is operationally negligible ($2Dd < 1$ s), DNS's $O(1)$ publication and churn costs and $O(M)$ centralized state yield strictly lower overhead than BGP.
- H2:** *BGP dominates in high-latency regimes.* When $2Dd$ exceeds tens of seconds, the cumulative per-query DNS cost for Q queries ($Q \cdot 2Dd$) rapidly exceeds BGP's one-time convergence cost (Dd), making BGP preferable after as few as one query.
- H3:** *A crossover point exists near cislunar delays.* For one-way delays around 1–2 s (characteristic of Earth–Moon links), neither protocol clearly dominates: DNS retains an overhead advantage while BGP provides lower amortized latency.

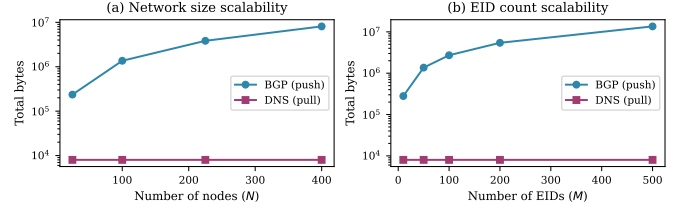


Fig. 2. Scalability of network overhead. (a) BGP overhead grows super-linearly with node count ($O(N^2)$ in a grid), while DNS remains constant. (b) BGP overhead scales linearly with EID count ($O(N \cdot M)$), while DNS overhead depends only on the query count.

H4: *DNS accuracy degrades with the convergence-to-churn ratio.* When $R = T_{\text{conv}}/\Delta_{\text{churn}} > 1$, DNS cache invalidation cannot keep pace with EID changes, reducing answer accuracy. BGP maintains correctness regardless of R due to incremental update propagation.

IV. EVALUATION

This section presents a comprehensive evaluation of the BGP-like and DNS-like protocols through 13 experiments organized in three groups: synthetic grid experiments (Sections IV-A–IV-D), deep-space experiments (Section IV-E), and realistic deployment scenarios (Section IV-F). All experiments use `dnsDirectMode=true` to ensure a fair comparison, modeling DNS over a pre-routed IP underlay as described in Section III-E.

A. Baseline Overhead and Latency

We first characterize the fundamental overhead and latency behavior of both protocols on a 10×10 grid (100 nodes, $d = 10$ ms) with 50 EIDs and no churn.

Overhead. BGP generates 1,355 KB of control traffic to flood announcements to all 100 nodes, compared to 9.6 KB for DNS (100 client queries $\times$ 80-byte responses). This represents a $141\times$ overhead ratio. The disparity arises from BGP's $O(N)$ message complexity per EID publication versus DNS's $O(1)$ per query.

State distribution. BGP replicates the full routing table at every node (100 nodes $\times$ 50 EIDs = 5,000 entries network-wide), while DNS stores records only at the authority (50 entries) plus a resolver cache of bounded size.

Discovery latency (Exp. 4). BGP converges in 0.20 s (18 hops $\times$ 10 ms), after which all lookups are instantaneous local table checks. DNS resolves queries in 0.25 s per round-trip. The critical distinction is that BGP's convergence cost is *one-time*, while DNS pays its query latency on *every resolution*.

B. Scalability

We evaluate how overhead scales along two dimensions: network size (Exp. 2) and EID count (Exp. 3). Fig. 2 shows the results.

Network size (Fig. 2a). As the grid grows from 5×5 (25 nodes) to 20×20 (400 nodes), BGP overhead increases from 6.6 KB to 197.8 KB—a $30\times$ increase for a $16\times$ growth

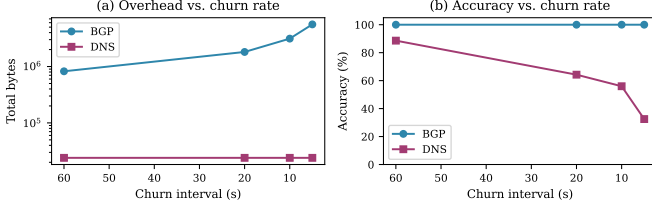


Fig. 3. Churn resilience. (a) BGP overhead scales inversely with churn interval (11–208 $\times$ more than DNS). (b) BGP maintains 100% accuracy at all churn rates; DNS accuracy degrades to 30% at high churn (5 s interval) with 60 s TTL.

in node count, confirming the super-linear $O(N^2)$ scaling in grid topologies. DNS overhead remains constant at ~ 8 KB, depending only on the query count.

EID count (Fig. 2b). On a fixed 10×10 grid, increasing EIDs from 10 to 500 causes BGP overhead to grow from 280 KB to 13.56 MB (~ 27 KB per EID $\times$ 100 nodes). DNS overhead remains at 8 KB. The BGP network-wide state grows as $O(N \cdot M)$, reaching 50,000 entries at 500 EIDs versus 500 entries for DNS authority storage.

C. Churn Resilience

Experiments 5 and 6 evaluate both protocols under EID churn (updates and withdrawals) on the 10×10 grid with 20 EIDs. Fig. 3 shows overhead and accuracy as a function of churn rate.

Overhead (Fig. 3a). At 5 s churn interval, BGP generates 5.0 MB of update traffic (208 $\times$ more than DNS’s 24 KB). Every EID change triggers a network-wide flood of UPDATE and WITHDRAW messages. DNS overhead remains constant at 24 KB (300 queries $\times$ 80 bytes), since churn only modifies the authority’s local database.

Accuracy (Fig. 3b). BGP maintains 100% answer accuracy at all churn rates, because UPDATE propagation keeps all routing tables current. DNS accuracy degrades under high churn: at 5 s churn with 60 s TTL, only 30.4% of cached responses are correct. At 60 s churn (matching the TTL), DNS recovers to 100% accuracy.

Staleness vs. TTL. Experiment 6 sweeps DNS TTL values under 10 s churn. Fig. 4a shows that shorter TTLs improve accuracy (83% at TTL=15 s vs. 35% at TTL=120 s) at the cost of increased query load. The optimal TTL should approximate the expected churn interval.

D. DNS Cache Effectiveness

Experiment 7 evaluates DNS cache performance under Zipf-distributed query patterns with parameter $\alpha \in \{0, 0.5, 1.0, 1.5, 2.0\}$ (Fig. 4b). With uniform queries ($\alpha = 0$), the cache achieves only a 0.5% hit rate. At $\alpha = 1.0$ (typical web traffic [19]), the hit rate reaches 16.9%, and at $\alpha = 2.0$ it rises to 39.4%. DNS caching thus provides a significant benefit primarily under skewed access patterns common in real networks.

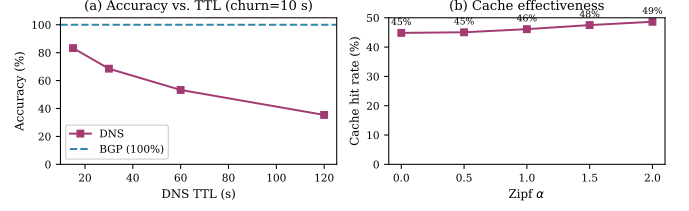


Fig. 4. DNS-specific analysis. (a) Answer accuracy versus TTL under 10 s churn; BGP (dashed) maintains 100%. (b) Cache hit rate increases with Zipf skewness α .

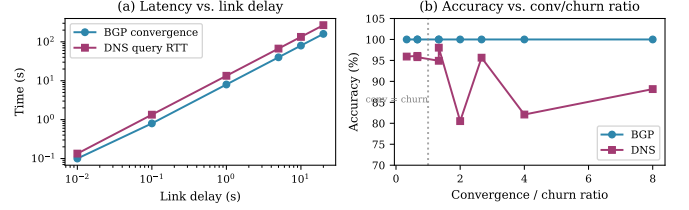


Fig. 5. Deep-space experiments. (a) Both BGP convergence and DNS query latency scale linearly with link delay; DNS pays $2 \times D \times d$ per query vs. BGP’s one-time $D \times d$. (b) DNS accuracy degrades when the convergence/churn ratio exceeds 1 (vertical line); BGP maintains 100%.

E. Deep-Space Scenarios

Experiments 8–10 explore protocol behavior on a 5×5 grid (diameter $D = 8$ hops) with link delays ranging from 10 ms to 20 s, representative of terrestrial through deep-space environments. Fig. 5 summarizes the key results.

Latency scaling (Fig. 5a). Both BGP convergence time and DNS query RTT scale linearly with link delay. At $d = 20$ s, BGP converges in 160 s (one-time cost) while each DNS query takes 400 s round-trip. For any workload with more than one query, BGP’s “pay once, use forever” model provides strictly lower cumulative latency.

DNS caching at high delay (Exp. 9). With Zipf $\alpha = 1.0$ caching and 300 s TTL, DNS achieves a consistent 18.7% cache hit rate (independent of link delay), yielding $\sim 12\%$ average latency reduction. At $d = 20$ s, this saves 46 s per 100 queries—helpful, but DNS still averages 354 s per query versus BGP’s 0 s after convergence.

Churn at high delay (Fig. 5b). The convergence-to-churn ratio $R = T_{\text{conv}}/\Delta_{\text{churn}}$ governs DNS accuracy. When $R < 1$ (churn slower than convergence), DNS maintains 95–98% accuracy. When $R > 2$, DNS degrades to 80–88%. BGP maintains 100% accuracy at all ratios because incremental updates propagate even when global convergence has not completed.

F. Realistic Deployment Scenarios

Beyond synthetic grid topologies, we evaluate three realistic DTN deployment scenarios spanning the latency spectrum from terrestrial to interplanetary networks. Each scenario uses a hierarchical topology with heterogeneous link delays, depicted in Fig. 6. Table III summarizes the key parameters.

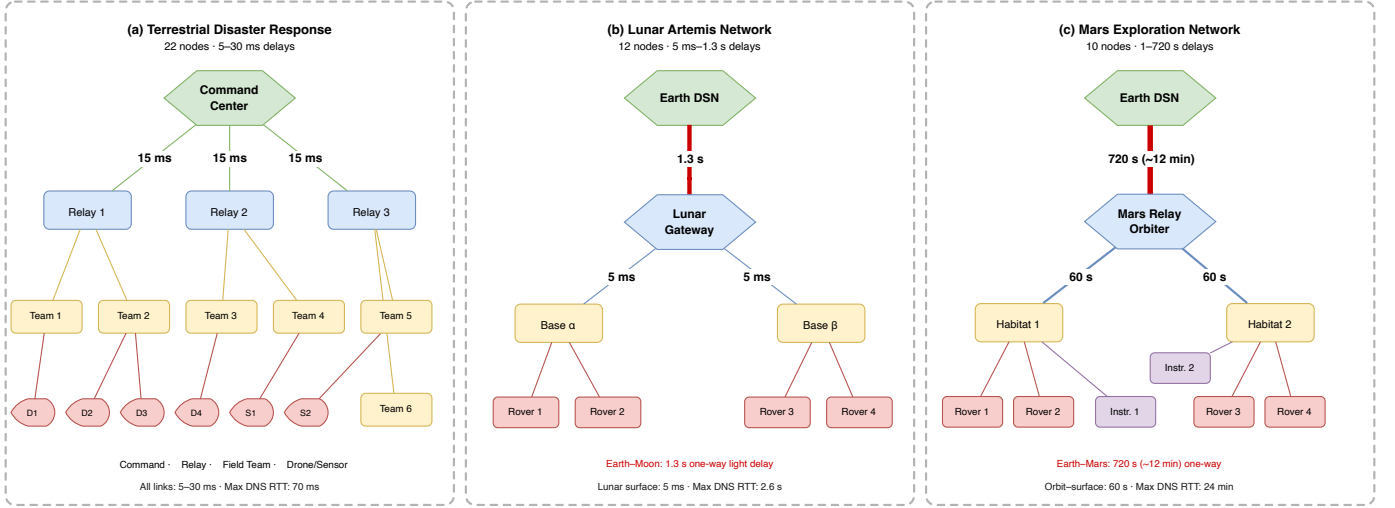


Fig. 6. Realistic deployment topologies. (a) Terrestrial disaster response: 22 nodes in a 4-tier hierarchy (command center, mobile relays, field teams, drones/sensors) with 5–30 ms links. (b) Lunar Artemis network: 12 nodes spanning Earth and Moon; the 1.3 s Earth–Moon link (bold) dominates latency. (c) Mars exploration: 10 nodes with a 720 s (~12 min) interplanetary link between Earth DSN and the Mars Relay Orbiter; orbit-to-surface links add 60 s each.

TABLE III
REALISTIC SCENARIO SUMMARY

Parameter	Terrestrial	Lunar	Mars
Nodes	22	12	10
Link delays	5–30 ms	5 ms–1.3 s	1–720 s
Max DNS RTT	70 ms	2.6 s	24 min
Churn interval	30–60 s	120 s	300 s
DNS TTL	30 s	60 s	120 s

1) *Terrestrial Disaster Response (Exp. 11)*: This scenario models a disaster-response network with 22 nodes in a 4-tier hierarchy: a central Command Center, 3 mobile relay vehicles, 6 field teams, and 12 drones/sensors, with link delays of 5–30 ms. Links are modeled as stable delay channels, representing a network in which IP connectivity between tiers has already been established—for example, via deployed mesh radios or satellite uplinks. The scenario therefore asks: *given that the field network is operational, which EID resolution paradigm is more efficient?* It does not model the disruptions and store-carry-forward behavior that DTN is designed to handle at a lower layer. In practice, disaster-response networks may also exhibit intermittent links (e.g., mobile units moving out of radio range), which would affect both protocols; that aspect is not captured here.

Fig. 7a shows the overhead results. Under 30 s churn, BGP generates 212.8 KB (13× more than DNS’s 16.4 KB). Under 60 s churn, the ratio drops to 9× (120.1 KB vs. 12.9 KB). DNS query latency averages 188 ms, which is operationally acceptable for disaster response. Both protocols achieve near-perfect accuracy ($\geq 96.7\%$). The reduction in overhead makes DNS the preferred choice for this scenario.

2) *Lunar Artemis Network (Exp. 12)*: The lunar scenario comprises 12 nodes spanning Earth and Moon: an Earth DSN ground station, a Lunar Gateway orbital relay, 2 surface bases, 4 rovers, and 4 field assets. The 1.3 s Earth–Moon

one-way delay creates an interesting tradeoff. As with the terrestrial scenario, all links are modeled as stable delay channels; the Earth–Moon and intra-lunar links are assumed to be continuously available. In a real Artemis deployment, Earth–Moon visibility windows are intermittent and surface rovers operate with duty-cycled radios; these factors would reduce the effective query frequency and may shift the break-even point between the two protocols.

Fig. 7b shows the amortized latency cost. BGP converges in ~ 1.6 s (a one-time cost), while DNS pays ~ 0.6 s per lunar-local query and ~ 2.8 s for Earth-involved queries. The *break-even point* occurs at approximately 2.7 queries: beyond this, BGP’s proactive model amortizes its convergence cost. DNS uses $3.8\times$ less bandwidth (6.3 KB vs. 23.8 KB), and both protocols achieve 100% accuracy with 120 s churn. This contested regime suggests that a hybrid approach—DNS for infrequent lunar-local queries, BGP for Earth-involved services—may be optimal.

3) *Mars Exploration Network (Exp. 13)*: The Mars scenario models 10 nodes with a 12-minute average Earth–Mars one-way delay: an Earth DSN station, a Mars Relay Orbiter, 2 surface habitats, 4 rovers, and 2 deployed instruments. Consistent with the other scenarios, all links are modeled as stable delay channels with fixed one-way propagation delays (720 s Earth–Mars, 60 s orbit-to-surface). Real Mars operations involve highly intermittent Earth–Mars visibility (typically a few hours per day) and relay-dependent surface connectivity; under such a contact schedule, BGP convergence would be further deferred to the next available contact window, while DNS queries would be queued and serviced on the same schedule. The stable-link assumption thus provides a best-case latency baseline for both protocols; intermittent contacts would increase absolute values but are not expected to alter the qualitative ranking.

Fig. 7c shows the cumulative query time. BGP converges

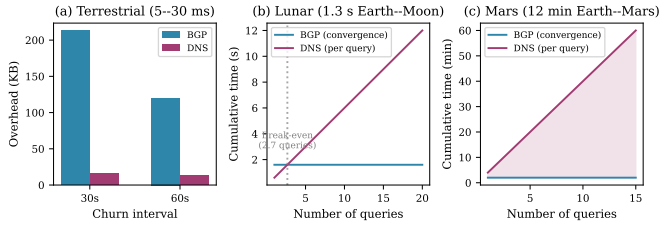


Fig. 7. Realistic deployment scenarios. (a) Terrestrial disaster response: DNS achieves 9–13 $\times$ lower overhead. (b) Lunar network: BGP amortizes convergence after ~ 3 queries. (c) Mars network: DNS cumulative query time grows linearly while BGP remains flat after convergence.

TABLE IV
SUMMARY OF KEY RESULTS ACROSS ALL EXPERIMENTS

Experiment	BGP	DNS	Winner
1: Baseline overhead	1,355 KB	9.6 KB	DNS
2: 400-node grid	197.8 KB	8 KB	DNS
3: 500 EIDs	13.56 MB	8 KB	DNS
4: Discovery latency	0.18 s	0.25 s	BGP
5: Churn accuracy (5 s)	100%	30.4%	BGP
6: Staleness (TTL=15 s)	100%	83.1%	BGP
7: Cache effectiveness	—	16.9% hit ($\alpha=1$)	—
8: Deep-space (20 s)	160 s once	400 s/query	BGP
9: Caching at high delay	—	18.7% hit	—
10: Deep churn ($R=8$)	100%	88.2%	BGP
11: Terrestrial	212.8 KB	16.4 KB	DNS
12: Lunar	23.8 KB	6.3 KB	Contested
13: Mars	2 min once	4 min/query	BGP

in ~ 2 minutes (one-time), after which all lookups are instantaneous. DNS pays ~ 4 minutes per Mars-local query and ~ 25 minutes for Earth-involved queries. After just 10 queries, DNS accumulates 40 minutes of total delay versus BGP’s fixed 2 minutes. DNS overhead is $3.4\times$ lower (5.3 KB vs. 18.0 KB), but this advantage is irrelevant when each query takes minutes. BGP is the only viable option for Mars-distance networks.

G. Cross-Scenario Analysis

Fig. 9 illustrates the protocol latency as a function of one-way link delay, highlighting the three deployment regimes. Fig. 8 synthesizes these regimes into a delay-driven decision map, linking each band to the confirmed hypothesis and realistic scenario. Table IV summarizes the key quantitative findings across all 13 experiments.

Three deployment regimes emerge from the data, confirming the hypotheses from Section III-I:

Terrestrial (<100 ms RTT)—H1 confirmed: DNS is preferred. Overhead savings of 9–141 $\times$ dominate, query latency (<200 ms) is operationally acceptable, and centralized state simplifies management under mobility.

Cislunar (100 ms–3 s RTT)—H3 confirmed: Contested. DNS retains an overhead advantage ($3.8\times$), but BGP amortizes convergence cost after ~ 3 queries. A hybrid strategy—DNS for infrequent local queries, BGP for critical and Earth-involved services—may be optimal.

Deep-space (>3 s RTT)—H2 confirmed: BGP is required. DNS per-query latency becomes prohibitive (minutes

to hours). Even with caching ($\sim 12\%$ latency reduction), DNS cannot compete with BGP’s instant post-convergence lookups. The break-even point occurs after a single query at Mars distances.

Hypothesis H4 is confirmed across all deep-space churn experiments (Fig. 5b): DNS accuracy degrades from $\sim 98\%$ to $\sim 80\%$ as the convergence-to-churn ratio R increases from 0.33 to 8.0, while BGP maintains 100% accuracy throughout.

V. CONCLUSIONS

This paper presented the first systematic comparison of push (BGP-like) and pull (DNS-like) paradigms for distributing EID reachability in Delay-Tolerant Networks, spanning one-way delays from 5 ms to 12 min across 13 experiments on synthetic and realistic topologies. All four hypotheses formulated in Section III-I were confirmed:

H1—DNS dominates at low latency. In terrestrial regimes ($2Dd < 1$ s), DNS achieves 9–141 $\times$ lower overhead than BGP with sub-200 ms query latency. Its $O(1)$ publication cost and centralized $O(M)$ state make it the clear choice for general, multi-hop EID resolution in bandwidth-constrained disaster-response and tactical DTN deployments. We emphasize that this conclusion applies to the generalized flooding model evaluated here; the BGP extension of [6] was originally designed for the more constrained problem of edge gateway-to-BPA configuration, which is structurally different from network-wide multi-hop flooding and is not directly addressed by this comparison.

H2—BGP dominates at high latency. Beyond ~ 3 s RTT, BGP’s one-time convergence cost ($D \cdot d$) amortizes after a single query. At Mars distances, each DNS resolution takes ~ 25 min for Earth-involved lookups, while BGP converges once in ~ 2 min and serves all subsequent lookups instantaneously. BGP is the only viable option for interplanetary networks.

H3—A crossover exists at cislunar delays. At Earth–Moon distances (~ 1 –3 s RTT), the two paradigms are genuinely contested: DNS retains a $3.8\times$ overhead advantage, but BGP amortizes its convergence cost after just ~ 3 queries. This regime calls for hybrid architectures—DNS for infrequent, latency-tolerant local queries and BGP for critical or Earth-involved services.

H4—DNS accuracy degrades under churn. When the convergence-to-churn ratio $R = T_{\text{conv}}/\Delta_{\text{churn}}$ exceeds 1, DNS cache staleness drives accuracy down—from 98% at $R = 0.33$ to 80% at $R = 8$ in deep-space experiments, and as low as 30% at 5 s terrestrial churn with 60 s TTL. BGP maintains 100% accuracy at all churn rates due to incremental update propagation.

These results yield a practical design rule: *the one-way link delay d is the single most predictive variable for protocol selection.* Network operators can use the delay-driven regime map (Figs. 9 and 8) to choose the appropriate paradigm—or identify the need for a hybrid—without detailed traffic modeling.

Several directions remain open. First, the hybrid cislunar strategy identified by H3 deserves a concrete protocol design that dynamically switches between push and pull based

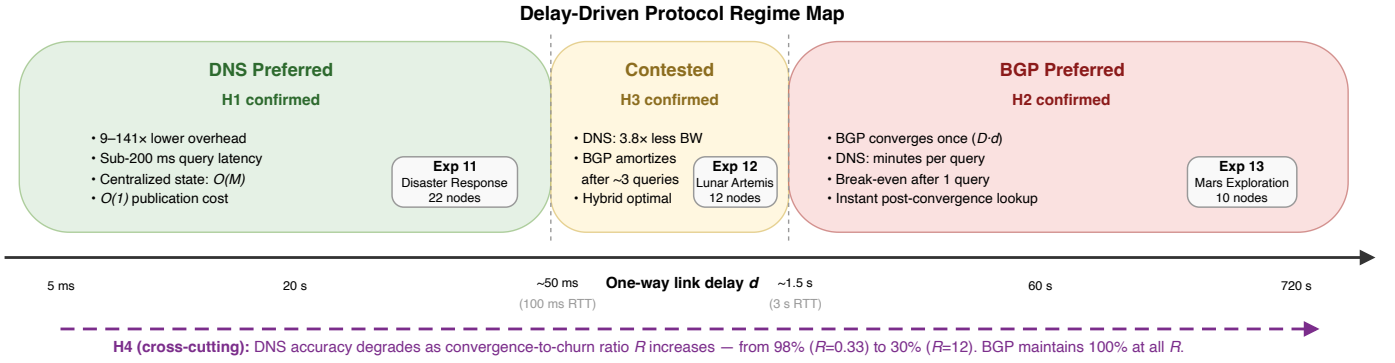


Fig. 8. Delay-driven protocol regime map. Three regimes emerge along the one-way link delay axis: DNS preferred (<50 ms, H1), contested/hybrid (50 ms–1.5 s, H3), and BGP preferred (>1.5 s, H2). Each band is annotated with the key performance indicators from the corresponding experiments. The cross-cutting arrow (H4) indicates that DNS accuracy degrades with the convergence-to-churn ratio R across all regimes, while BGP maintains 100% accuracy regardless of R .

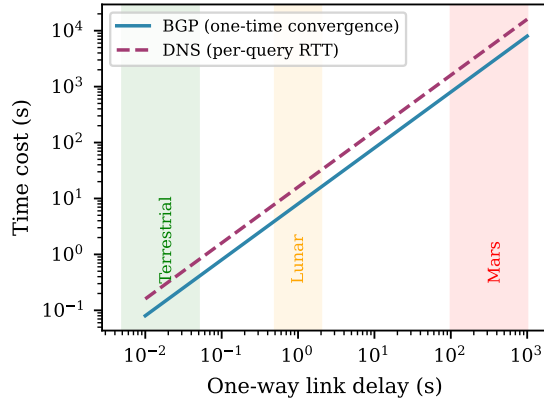


Fig. 9. Protocol latency versus link delay. BGP convergence (one-time, $D \cdot d$) grows slower than DNS per-query RTT ($2 \cdot D \cdot d$). Shaded regions indicate terrestrial, lunar, and Mars delay regimes.

on query frequency and link conditions. Second, our DNS model uses a single authority tier; a hierarchical authority chain (mirroring the terrestrial root/TLD/authoritative structure) could reduce worst-case query latency in multi-domain space networks. Third, the interaction between EID resolution and contact-plan-based routing [21] in scheduled DTN links is unexplored and may further shift the push-pull boundary. Finally, integrating security mechanisms such as BPsec [22] into both paradigms will introduce additional overhead that may alter the crossover points identified here.

ACKNOWLEDGMENT

This work was funded by the DARKSOL project (<https://darksol.cloud/en/>), co-financed by the project partners, the European Union, and the Free State of Saxony.

REFERENCES

- [1] K. Fall, “A delay-tolerant network architecture for challenged internets,” in *Proceedings of the 2003 conference on Applications, technologies, architectures, and protocols for computer communications*. ACM, 2003, pp. 27–34.
- [2] S. Burleigh, K. Fall, and E. Birrane, “Bundle Protocol Version 7,” RFC 9171, January 2022.
- [3] V. Cerf, S. Burleigh, A. Hooke, L. Torgerson, R. Durst, K. Scott, K. Fall, and H. Weiss, “Delay-Tolerant Networking Architecture,” RFC 4838, April 2007.
- [4] D. J. Israel, K. D. Mauldin, C. J. Roberts, J. W. Mitchell, A. A. Pulkkinen, L. V. A. Cooper, M. A. Johnson, S. D. Christe, and C. J. Gramling, “LunaNet: A Flexible and Extensible Lunar Exploration Communications and Navigation Infrastructure,” in *2020 IEEE Aerospace Conference*. IEEE, 2020, pp. 1–14.
- [5] A. Alhilal, T. Braud, and P. Hui, “The sky is not the limit anymore: Future architecture of the interplanetary internet,” *IEEE Aerospace and Electronic Systems Magazine*, vol. 34, no. 8, pp. 22–32, 2019.
- [6] M. Feldmann, T. Tchilinguirian, and F. Walter, “A Border Gateway Protocol Extension for Distributing Endpoint Identifier Reachability Information in Delay-tolerant Networks,” in *IEEE International Conference on Wireless for Space and Extreme Environments (WiSEE)*. IEEE, 2025, in Press.
- [7] E. Kline, “The ipn.arpa Zone and IPN DNS Operations,” Internet-Draft draft-ek-dtn-ipn-arpa-00, November 2025, work in Progress.
- [8] D. Project, “bgp-dns-omnetpp: Omnet++ models for dns vs. bgp eid reachability comparison,” 2026, accessed: 2026-02-15. [Online]. Available: <https://github.com/darksol-cloud/bgp-dns-omnetpp>
- [9] S. Burleigh, A. Hooke, L. Torgerson, K. Fall, V. Cerf, B. Durst, and K. Scott, “Delay-Tolerant Networking: An Approach to Interplanetary Internet,” *IEEE Communications Magazine*, vol. 41, no. 6, pp. 128–136, 2003.
- [10] K. Scott and S. Burleigh, “Bundle Protocol Specification,” RFC 5050, November 2007.
- [11] B. Sipos, M. Demmer, J. Ott, and S. Perreault, “Delay-Tolerant Networking TCP Convergence-Layer Protocol Version 4,” RFC 9174, January 2022.
- [12] CCSDS, “CCSDS Bundle Protocol Specification,” Consultative Committee for Space Data Systems, Washington, D.C., Tech. Rep. CCSDS 734.2-B-1, 2015.
- [13] C. Caini, H. Cruickshank, S. Farrell, and M. Marchese, “Delay-and disruption-tolerant networking (dtn): An alternative solution for future satellite networking applications,” *Proceedings of the IEEE*, vol. 99, no. 11, pp. 1980–1997, 2011.
- [14] J. A. Fraire, O. De Jonckère, and S. C. Burleigh, “Routing in the Space Internet: A contact graph routing tutorial,” *Journal of Network and Computer Applications*, vol. 173, p. 102884, 2021.
- [15] Y. Rekhter, T. Li, and S. Hares, “A Border Gateway Protocol 4 (BGP-4),” RFC 4271, January 2006.
- [16] C. Labovitz, A. Ahuja, A. Bose, and F. Jahanian, “Delayed internet routing convergence,” *IEEE/ACM Transactions on Networking*, vol. 9, no. 3, pp. 293–306, 2001.
- [17] P. Mockapetris, “Domain Names - Implementation and Specification,” RFC 1035, November 1987.
- [18] J. Jung, E. Sit, H. Balakrishnan, and R. Morris, “DNS Performance and the Effectiveness of Caching,” *IEEE/ACM Transactions on Networking*, vol. 10, no. 5, pp. 589–603, 2002.
- [19] L. Breslau, P. Cao, L. Fan, G. Phillips, and S. Shenker, “Web caching and zipf-like distributions: Evidence and implications,” in *IEEE INFOCOM’99. Conference on Computer Communications. Proceedings*.

- Eighteenth Annual Joint Conference of the IEEE Computer and Communications Societies*, vol. 1. IEEE, 1999, pp. 126–134.
- [20] A. Varga, “OMNeT++,” in *Modeling and Tools for Network Simulation*. Springer, 2010, pp. 35–59.
 - [21] G. Araniti, N. Bezirgiannidis, E. Birrane, I. Bisio, S. Burleigh, C. Caini, M. Feldmann, M. Marchese, J. Segui, and K. Suzuki, “Contact graph routing in dtn space networks: Overview, enhancements and performance,” *IEEE Communications Magazine*, vol. 53, no. 3, pp. 38–46, 2015.
 - [22] E. B. III and K. McKeever, “Bundle Protocol Security (BPsec),” RFC 9172, January 2022.